

NAFNet FPGA Hardware Accelerator

Complete Project Summary — Software, Quantization, Hardware Architecture, RTL Plan and FPGA Deployment

Scope: Train NAFNet for grayscale image restoration, quantize it, implement the inference datapath as an FPGA accelerator, and evaluate restoration quality, latency and FPGA resources.

1. Problem Definition

The model receives a degraded grayscale image and reconstructs the corresponding clean, full-resolution image. Required degradations are speckle noise, Gaussian noise and spatial-resolution reduction. Multiple degradations may occur simultaneously. The test set includes in-distribution and out-of-distribution samples, so generalization is required. Inference speed is also benchmarked.

Degradation	What it looks like	What the model must fix
Speckle noise	Random pixel-level grain; values can exceed the normal range.	Remove noise while preserving genuine detail; avoid destructive blurring.
Gaussian noise	Soft/hazy appearance; edges and fine structures lose sharpness.	Restore edge sharpness and contrast without artificial patterns or ringing.
Spatial resolution reduction	Downsampled input, e.g. 512→256 or 256→128.	Upscale to the original resolution and reconstruct lost details.

2. Dataset

Item	Size / count	Role
Training GT	3,200 × 256×256	Clean full-resolution target
Training NoisyLR	3,200 × 128×128	Degraded input
Train / validation	2,880 / 320	Training and validation; no train/validation leakage
Test / OOD NoisyLR	400 × 128×128	Held-out evaluation

Observed training sample: NoisyLR = [1,128,128], float32, approximately −0.0053 to 1.6235. GT = [1,256,256], float32, 0 to 1.

The 400 test files were valid 128×128 float32 images. A filename-overlap warning was observed, but the images were subsequently checked and confirmed to be completely different images.

3. NAFNet Software Model

NAFNet is the restoration backbone. It maps a 128×128 single-channel degraded image to a 256×256 single-channel restored image. The model contains eight NAFBlocks followed by reconstruction and ×2 spatial upscaling.

Software path: NoisyLR 128×128 → feature extraction → 8 NAFBlocks → reconstruction → PixelShuffle ×2 → final restoration → 256×256 output.

4. Training Results

Metric	Best result
Best epoch	20
Validation L1	0.035449
PSNR	27.037 dB
SSIM	0.7170
Baseline L1	0.040944
Baseline PSNR	24.819 dB
Baseline SSIM	0.6715

5. Test / OOD Inference

All 400 held-out images were processed successfully. Raw model outputs were clamped to [0,1] for final restored-image representation. Reported processing time for the 400-image software pass was 10.094 seconds.

6. NAFNet Computational Analysis

Property	Result
Input	1 × 128 × 128
Output	1 × 256 × 256

Property	Result
Convolution layers	52
Convolution parameters	74,990
Total model parameters	76,526
Total MACs / image	1,055,989,760
Total compute	1.056 GMAC / image
Standard convolution MACs	975,183,872 (92.35%)
Depthwise MACs	80,805,888 (7.65%)
INT8 weight memory estimate	≈74.73 KB

7. Memory Analysis

Data	FP32	FP16 / INT16	INT8
76,526 parameters	298.93 KB	149.46 KB	74.73 KB
32×128×128 feature map	2.00 MB	1.00 MB	512 KB
64×128×128 feature map	4.00 MB	2.00 MB	1.00 MB
64×32×32 tile	—	128 KB	64 KB

Three-row 3×3 line buffers require about 24 KB INT16 for 32 channels and 48 KB INT16 for 64 channels. This supports streaming/tiling rather than full feature-map buffering.

8. Quantization Strategy

The starting hardware precision is INT8 weights, INT16 activations, INT24 products and INT32 accumulation. Layer-wise/tensor-wise scaling is used because activation ranges vary substantially between layers.

Element	Format	Purpose
Weights	INT8	Reduce storage and multiplication cost
Activations	INT16	Provide additional numerical range
Product	INT24	INT16 $\times$ INT8 multiplication result
Accumulator	INT32	Safe convolution accumulation
Output activation	INT16	Requantized value for the next stage

Calibration examples: one layer was approximately -163.34 to $+154.05$; another -15.40 to $+19.17$; the ending stage about -3.16 to $+3.78$; and the final stage about -0.112 to $+1.211$. This confirms that one global activation scale is unsuitable.

9. Fixed-Point Accumulator Analysis

Operation	Products/output	Worst-case magnitude	INT32 safe
1 $\times$ 1 32 $\rightarrow$ 64	32	133,165,088	YES
1 $\times$ 1 32 $\rightarrow$ 32	32	133,165,088	YES
3 $\times$ 3 32 $\rightarrow$ 32	288	1,198,485,792	YES
3 $\times$ 3 depthwise	9	37,452,681	YES
3 $\times$ 3 ending	288	1,198,485,792	YES
3 $\times$ 3 up	288	1,198,485,792	YES
3 $\times$ 3 final	9	37,452,681	YES

Proposed datapath: INT16 activation $\times$ INT8 weight $\rightarrow$ INT24 product $\rightarrow$ INT32 accumulation $\rightarrow$ requantization $\rightarrow$ INT16 activation.

10. FPGA Parallelism

MAC lanes	Cycles / image	Theoretical latency @200 MHz	Images/s
32	32,999,680	165.00 ms	6.06
64	16,499,840	82.50 ms	12.12
128	8,249,920	41.25 ms	24.24
256	4,124,960	20.62 ms	48.49

Selected starting target: 64 parallel MAC lanes. Theoretical latency excludes memory, control and other implementation overhead; actual FPGA latency must be measured after implementation.

11. Tile / BRAM Architecture

Initial design target: 32 $\times$ 32 spatial tiles, up to 64 INT16 channels, 3-row line buffers, and a reusable 64-MAC engine. A 64 $\times$ 32 $\times$ 32 INT16 tile requires 128 KB; the 64-channel 3-row line buffer requires about 48 KB.

Dataflow: input/feature buffer $\rightarrow$ 3 $\times$ 3 line buffers $\rightarrow$ reusable 64-MAC engine $\rightarrow$ INT32 accumulation $\rightarrow$ requantization $\rightarrow$ INT16 storage $\rightarrow$ NAFBlock operations $\rightarrow$ reconstruction $\rightarrow$ PixelShuffle $\times 2 \rightarrow$ output buffer.

12. Layer-Level Hardware Schedule

Stage	Operation	Spatial size	MACs
Input	1 $\rightarrow$ 32, 3 $\times$ 3	128 $\times$ 128	4.719 M
Each NAFBlock	32 $\rightarrow$ 64, 1 $\times$ 1	128 $\times$ 128	33.554 M
Each NAFBlock	64 $\rightarrow$ 64 depthwise, 3 $\times$ 3	128 $\times$ 128	9.437 M
Each NAFBlock	32 $\rightarrow$ 32, 1 $\times$ 1	128 $\times$ 128	16.777 M
Each NAFBlock	32 $\rightarrow$ 64, 1 $\times$ 1	128 $\times$ 128	33.554 M
Each NAFBlock	32 $\rightarrow$ 32, 1 $\times$ 1	128 $\times$ 128	16.777 M
Ending	32 $\rightarrow$ 32, 3 $\times$ 3	128 $\times$ 128	150.995 M
Up	32 $\rightarrow$ 4, 3 $\times$ 3	128 $\times$ 128	18.874 M
Final	1 $\rightarrow$ 1, 3 $\times$ 3	256 $\times$ 256	0.590 M

The full schedule contains 52 convolution layers and 1.056 GMAC/image. The ending 3×3 convolution is the largest individual convolution workload at approximately 150.995 M MACs.

13. FPGA Accelerator — Actual Project Goal

The FPGA is being used as a genuine hardware accelerator for the complete quantized NAFNet inference path. Python/Jupyter is not intended to replace FPGA inference; it is used for input preparation, control, DMA transfers, visualization and evaluation.

Final hardware path: degraded 128×128 image → PS/DDR → AXI DMA → FPGA NAFNet accelerator → 256×256 restored output → DDR → Jupyter.

FPGA datapath: input buffer → convolution engine → NAFBlocks ×8 → reconstruction → PixelShuffle ×2 → final convolution → output buffer.

14. RTL Module Plan

Order	Module	Role
1	mac_unit	INT16×INT8 multiplication and INT32 accumulation
2	mac_array	64 parallel MAC lanes
3	line_buffer	Streaming 3×3 neighborhood generation
4	conv_engine	1×1, 3×3 and depthwise convolution
5	groupnorm	NAFNet normalization
6	simple_gate	NAFNet SimpleGate
7	channel_attention	Channel-attention path
8	residual_add	Residual feature addition
9	naf_block	Complete NAFBlock
10	pixel_shuffle	×2 spatial rearrangement
11	nafnet_accelerator	Complete NAFNet inference accelerator
12	AXI/control wrapper	PS/Jupyter configuration and DMA interface

15. Python Golden Model — Purpose

The Golden Model is a verification reference, not deployed inference hardware. It emulates the proposed integer arithmetic and provides expected results for RTL and FPGA comparison. Validated kernels include 1×1 standard convolution, 3×3 standard convolution and 3×3 depthwise convolution using INT16×INT8 arithmetic, INT32 accumulation and INT16 requantization.

Verification: Python Golden Model → expected integer output → RTL simulation → FPGA output → numerical comparison. Separately, FPGA output is compared against ground truth using L1, PSNR and SSIM.

16. Vivado → Bitstream → PYNQ-Z2

After RTL verification, the design is integrated in Vivado for the target Zynq-7000/PYNQ-Z2 device. Synthesis and implementation provide LUT, FF, DSP, BRAM, timing and power information. After successful implementation, the FPGA bitstream and hardware metadata are deployed to the PYNQ-Z2.

In Jupyter/PYNQ, the overlay loads the accelerator hardware. Buffers are allocated for the 128×128 INT16 input and 256×256 INT16 output. The input is placed in DDR, transferred through AXI DMA to the FPGA accelerator, processed entirely by the hardware NAFNet path, and returned through DMA to DDR. Jupyter then dequantizes, displays and evaluates the result.

17. Golden Model vs FPGA

Comparison	Purpose
Golden Model vs RTL	Verify arithmetic and datapath correctness
Golden Model vs FPGA	Verify synthesized hardware inference matches the reference
FPGA vs Ground Truth	Measure actual restoration quality
FP32 NAFNet vs FPGA	Measure accuracy loss from quantization/hardware
FPGA latency	Measure real inference time
FPGA resources	Measure LUT, FF, DSP, BRAM usage
Timing	Verify target operating frequency
Power	Estimate or measure accelerator power

18. End-to-End Project Flow

Dataset → NAFNet training → validation → OOD evaluation → model analysis → quantization calibration → INT8/INT16 design → Python Golden Model → RTL accelerator → RTL verification → Vivado synthesis → timing/resource analysis → implementation → bitstream → PYNQ-Z2 deployment → AXI DMA/Jupyter control → real FPGA inference → image-quality and performance comparison.

19. Current Status

Stage	Status
Dataset preparation and split	Completed
NAFNet training	Completed
Visual restoration check	Completed
Test/OOD processing	Completed
Model computational analysis	Completed
Activation calibration	Completed
Quantization strategy	Defined
Fixed-point analysis	Completed
Integer Golden Model kernels	Passed unit tests
FPGA architecture	Defined
RTL implementation	Next
Vivado implementation	Future
PYNQ-Z2 deployment	Future

Note: This PDF intentionally excludes implementation code. Final FPGA latency, resource utilization, timing closure and power are not yet measured and must be obtained after RTL/Vivado implementation.